TP 2 — Mettre en œuvre un pipeline MLOps simple

Durée : 3 h + 45 min de défis · Partie 2 du cours · binôme de 2

Un pipeline transforme une suite de commandes en un enchaînement décrit, paramétré et rejouable. Vous construisez ici un pipeline en trois étapes piloté par un fichier de paramètres, avec une règle de validation qui refuse d'enregistrer un modèle insuffisant. La séance se termine par des défis notés en bonus.

Objectifs pédagogiques

- Découper un traitement monolithique en étapes indépendantes et testables
- Externaliser tous les réglages dans un fichier de paramètres
- Structurer les exécutions MLflow en exécution parente et exécutions enfants
- Poser une règle de validation automatique avant l'enregistrement du modèle
- Rendre le pipeline exécutable par une seule commande, sur n'importe quel poste

Prérequis

- TP 1 terminé : environnement, données et serveur MLflow opérationnels
- Le serveur MLflow démarré sur le port 5000, dans un terminal laissé ouvert

Installation

terminal

```
cd tp-mlops && source .venv/bin/activate
pip install pyyaml matplotlib
mkdir -p src reports models
```

RÉSULTAT ATTENDU

```
Successfully installed PyYAML-6.x matplotlib-3.x (ou Requirement already satisfied)
```

Déroulement

Étape 1 — Externaliser les réglages

Aucune valeur numérique ne doit rester dans le code : c'est la condition pour qu'un outil puisse détecter qu'un réglage a changé.

params.yaml

```
cat > params.yaml << 'EOF'
preparation:
  test_size: 0.25            # part des donnees reservee a l'evaluation
  random_state: 42           # graine du decoupage
entrainement:
  n_estimators: 300          # nombre d'arbres
  max_depth: 12              # profondeur maximale
validation:
  seuil_f1: 0.82             # sous ce seuil, le modele est refuse
EOF
```

Étape 2 — Préparation

Cette étape lit les données brutes, effectue le découpage et écrit deux fichiers. Elle ne connaît rien du modèle.

src/preparer.py

```
cat > src/preparer.py << 'EOF'
import yaml, os, pandas as pd
from sklearn.model_selection import train_test_split
p = yaml.safe_load(open('params.yaml'))['preparation']    # lit les reglages
df = pd.read_csv('data/transactions.csv')
tr, te = train_test_split(df, test_size=p['test_size'],   # meme decoupage qu'aux TP 1 et 3
                          random_state=p['random_state'])
os.makedirs('data/prepare', exist_ok=True)
tr.to_csv('data/prepare/train.csv', index=False)          # sortie 1
te.to_csv('data/prepare/test.csv', index=False)           # sortie 2
print('preparation :', len(tr), 'lignes train /', len(te), 'lignes test')
EOF
python src/preparer.py
```

RÉSULTAT ATTENDU

```
preparation : 3750 lignes train / 1250 lignes test
```

CORRECTION PAR RAPPORT À LA VERSION PRÉCÉDENTE

La première version de cette fiche utilisait `stratify=df['fraude']`. La stratification est une bonne pratique sur des classes déséquilibrées, mais elle produit un découpage différent de celui des TP 1 et 3 : le F1 tombe alors à 0,7849, sous le seuil de 0,82, et le modèle est refusé à l'étape 5. On la retire ici pour garder un découpage identique d'un TP à l'autre, et donc des résultats comparables. La question 6 du compte rendu y revient.

Étape 3 — Entraînement

Cette étape ne lit que les fichiers produits par l'étape 1. Elle ne refait jamais le découpage : c'est ce qui garantit que l'évaluation porte toujours sur les mêmes lignes.

src/entrainer.py

```
cat > src/entrainer.py << 'EOF'
import yaml, pandas as pd, joblib
from sklearn.ensemble import RandomForestClassifier
p = yaml.safe_load(open('params.yaml'))['entrainement']
tr = pd.read_csv('data/prepare/train.csv')                # entree : sortie de l'etape 1
X, y = tr.drop(columns='fraude'), tr['fraude']
m = RandomForestClassifier(n_estimators=p['n_estimators'],
                           max_depth=p['max_depth'], random_state=42).fit(X, y)
joblib.dump(m, 'models/model.pkl')                        # sortie : l'artefact modele
print('entrainement : modele ecrit dans models/model.pkl')
EOF
python src/entrainer.py
```

RÉSULTAT ATTENDU

```
entrainement : modele ecrit dans models/model.pkl
```

Étape 4 — Évaluation

src/evaluer.py

```
cat > src/evaluer.py << 'EOF'
import json, joblib, pandas as pd
import matplotlib; matplotlib.use('Agg')          # pas de fenetre graphique
import matplotlib.pyplot as plt
from sklearn.metrics import f1_score, precision_score, recall_score, ConfusionMatrixDisplay
te = pd.read_csv('data/prepare/test.csv')
X, y = te.drop(columns='fraude'), te['fraude']
m = joblib.load('models/model.pkl')
pred = m.predict(X)
```

```
mets = {'f1': round(f1_score(y, pred), 4),
        'precision': round(precision_score(y, pred), 4),
        'recall': round(recall_score(y, pred), 4)}
json.dump(mets, open('reports/metrics.json', 'w'), indent=2)
ConfusionMatrixDisplay.from_predictions(y, pred)
plt.savefig('reports/confusion.png', dpi=110, bbox_inches='tight')
print('evaluation :', mets)
EOF
python src/evaluer.py
```

RÉSULTAT ATTENDU

```
evaluation : {'f1': 0.8636, 'precision': 0.8769, 'recall': 0.8507}
```

Étape 5 — Orchestrer les trois étapes avec une règle de validation

Le pipeline enchaîne les étapes, journalise le tout dans une exécution parente MLflow, puis n'enregistre le modèle que si la métrique dépasse le seuil.

pipeline.py

```
cat > pipeline.py << 'EOF'
import subprocess, json, yaml, joblib, mlflow, mlflow.sklearn
P = yaml.safe_load(open('params.yaml'))
mlflow.set_tracking_uri('http://127.0.0.1:5000')
mlflow.set_experiment('tp2-pipeline')
with mlflow.start_run(run_name='pipeline'):                # execution parente
    for etape in ['preparer', 'entrainer', 'evaluer']:
        with mlflow.start_run(run_name=etape, nested=True): # execution enfant
            subprocess.run(['python', f'src/{etape}.py'], check=True)
    mets = json.load(open('reports/metrics.json'))
    mlflow.log_params({**P['preparation'], **P['entrainement']})
    mlflow.log_metrics(mets)
    mlflow.log_artifact('reports/confusion.png')
    seuil = P['validation']['seuil_f1']
    if mets['f1'] >= seuil:                                # regle de validation
        mlflow.sklearn.log_model(joblib.load('models/model.pkl'), name='model',
                                 registered_model_name='fraude-tp2',
                                 skops_trusted_types=['sklearn.tree._tree.Tree'])
        print(f"modele accepte (F1 {mets['f1']} >= {seuil}) et enregistre")
    else:
        mlflow.set_tag('statut', 'refuse')
        raise SystemExit(f"modele refuse : F1 {mets['f1']} < seuil {seuil}")
EOF
python pipeline.py
```

RÉSULTAT ATTENDU

```
preparation : 3750 lignes train / 1250 lignes test
entrainement : modele ecrit dans models/model.pkl
evaluation : {'f1': 0.8636, 'precision': 0.8769, 'recall': 0.8507}
Successfully registered model 'fraude-tp2'.
Created version '1' of model 'fraude-tp2'.
modele accepte (F1 0.8636 >= 0.82) et enregistre
```

POURQUOI skops_trusted_types ?

Depuis MLflow 3.16, les modèles scikit-learn sont enregistrés au format sécurisé skops, qui refuse par défaut les structures internes d'une forêt aléatoire. Sans cette option, l'enregistrement échoue avec « untrusted types ». On déclare ici explicitement faire confiance au modèle que l'on vient d'entraîner soi-même.

Étape 6 — Vérifier que le garde-fou fonctionne

Un pipeline sans règle de validation laisse passer n'importe quoi. On dégrade volontairement le modèle pour vérifier que la chaîne s'arrête.

terminal

```
sed -i 's/max_depth: 12/max_depth: 2/' params.yaml     # modele volontairement trop simple
python pipeline.py                                     # doit echouer
echo "code de sortie : $?"                             # different de 0
sed -i 's/max_depth: 2/max_depth: 12/' params.yaml     # on remet la valeur correcte
```

RÉSULTAT ATTENDU

```
modele refuse : F1 0.2013 < seuil 0.82
code de sortie : 1

Dans MLflow, l'execution parente porte le tag statut=refuse et aucune nouvelle
version n'a ete enregistree.
```

Étape 7 — Rendre le pipeline portable

Le fichier MLproject permet de lancer le pipeline sur n'importe quelle machine. Attention : `mlflow run` crée lui-même l'exécution ; il faut donc lui indiquer le serveur et la même expérimentation que celle du script.

MLproject + terminal

```
cat > MLproject << 'EOF'
name: tp2-pipeline
entry_points:
  main:
    command: 'python pipeline.py'
EOF
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
mlflow run . --env-manager local --experiment-name tp2-pipeline
```

RÉSULTAT ATTENDU

```
Registered model 'fraude-tp2' already exists. Creating a new version of this model...
Created version '2' of model 'fraude-tp2'.
modele accepte (F1 0.8636 >= 0.82) et enregistre
=== Run (ID '...') succeeded ===
```

Si ça coince : sans `--experiment-name`, MLflow refuse avec « active experiment ID does not match environment run ID ». Sans `--env-manager local`, il tente de recréer un environnement conda.

Défis — 45 minutes, notés en bonus

Quatre défis indépendants. Le défi 1 est le défi principal ; les suivants sont destinés aux binômes rapides. Règles communes : il est interdit de modifier `src/preparer.py` et `test_size` (le jeu de test doit rester le même pour tous). Le classement final est recalculé par l'enseignant sur un jeu de test caché, qu'aucun binôme n'a vu.

Défi 1 — Le meilleur modèle… sous contraintes

Objectif : obtenir le meilleur F1 avec un modèle qui pèse au plus 2 Mo et prédit les 1 250 lignes de test au moins trois fois plus vite que le modèle de base, mesuré sur votre propre machine. Vous pouvez changer d'algorithme, de réglages, et transformer les variables dans `entrainer.py` et `evaluer.py`.

1. Mesurez d'abord le modèle de base :

src/mesurer.py

```
cat > src/mesurer.py << 'EOF'
import os, json, time, statistics, joblib, pandas as pd
m = joblib.load('models/model.pkl')
X = pd.read_csv('data/prepare/test.csv').drop(columns='fraude')
taille = os.path.getsize('models/model.pkl') / 1e6          # taille en Mo
m.predict(X)                                                # premier appel : chauffe
t = []
for _ in range(20):                                         # 20 mesures, on garde la mediane
    d = time.perf_counter(); m.predict(X); t.append((time.perf_counter() - d) * 1000)
latence = statistics.median(t)                              # ms pour les 1 250 lignes
json.dump({'taille_mo': round(taille, 2), 'latence_ms': round(latence, 1)},
          open('reports/mesures.json', 'w'), indent=2)
print(f'taille : {taille:.2f} Mo | latence : {latence:.1f} ms pour {len(X)} lignes')
EOF
python src/mesurer.py
```

RÉSULTAT ATTENDU

```
taille : 8.15 Mo | latence : 33.6 ms pour 1250 lignes
(la latence depend de votre machine : notez la votre)
```

2. Ajoutez les contraintes à `params.yaml` — la latence maximale vaut le tiers de celle que vous venez de mesurer :

params.yaml (ajout)

```
cat >> params.yaml << 'EOF'
defi:
  taille_max_mo: 2.0
  latence_max_ms: 11     # a remplacer par VOTRE latence / 3
EOF
```

3. Créez le pipeline du défi, qui journalise sur le serveur commun de la salle, puis lancez-le avec le nom de votre binôme :

pipeline_defi.py

```
cat > pipeline_defi.py << 'EOF'
import subprocess, json, yaml, sys, mlflow
BINOME = sys.argv[1]                              # ex. : python pipeline_defi.py binome-07
P = yaml.safe_load(open('params.yaml'))
mlflow.set_tracking_uri('http://IP_DU_PROF:5000')  # serveur commun de la salle
mlflow.set_experiment('defi-tp2')
with mlflow.start_run(run_name=BINOME):
    for etape in ['preparer', 'entrainer', 'evaluer', 'mesurer']:
        subprocess.run(['python', f'src/{etape}.py'], check=True)
    mets = json.load(open('reports/metrics.json'))
    mes = json.load(open('reports/mesures.json'))
    ok = (mes['taille_mo'] <= P['defi']['taille_max_mo']
          and mes['latence_ms'] <= P['defi']['latence_max_ms'])
    mlflow.log_params(P['entrainement'])
```

```
    mlflow.log_metrics({**mets, **mes})
    mlflow.set_tags({'binome': BINOME, 'contraintes': 'ok' if ok else 'hors-contraintes'})
    print(f"{BINOME} : F1 {mets['f1']} | {mes['taille_mo']} Mo | {mes['latence_ms']} ms"
          f" | contraintes {'OK' if ok else 'NON respectees'}")
EOF
python pipeline_defi.py binome-07
```

RÉSULTAT ATTENDU

```
binome-07 : F1 0.8636 | 8.15 Mo | 32.6 ms | contraintes NON respectees

Avec n_estimators: 60 et max_depth: 14 :
binome-07 : F1 0.8615 | 1.65 Mo | 7.6 ms | contraintes OK
```

Chaque tentative est une exécution MLflow : l'historique complet de votre démarche est visible par l'enseignant. Un indice pour les plus curieux : relisez la règle qui a généré les données au TP 1.

Défi 2 — Le seuil qui coûte le moins cher

Pour la banque, une fraude manquée coûte 1 000 MAD et une fausse alerte 50 MAD. Le seuil de 0,5 n'a aucune raison d'être le bon. Complétez le script pour trouver le seuil qui minimise le coût total.

src/seuil_cout.py

```
cat > src/seuil_cout.py << 'EOF'
import joblib, numpy as np, pandas as pd
from sklearn.metrics import confusion_matrix
COUT_FN, COUT_FP = 1000, 50          # fraude manquee / fausse alerte, en MAD
te = pd.read_csv('data/prepare/test.csv')
m = joblib.load('models/model.pkl')
proba = m.predict_proba(te.drop(columns='fraude'))[:, 1]    # probabilite de fraude
def cout(seuil):
    tn, fp, fn, tp = confusion_matrix(te['fraude'], (proba >= seuil).astype(int)).ravel()
    return COUT_FN * fn + COUT_FP * fp, fn, fp
print('seuil 0.50 :', cout(0.50))
# A COMPLETER : parcourir les seuils de 0.05 a 0.95 par pas de 0.01,
# trouver celui qui minimise le cout, et afficher cout, fraudes manquees, fausses alertes
EOF
python src/seuil_cout.py
```

RÉSULTAT ATTENDU

```
seuil 0.50 : (20800, 20, 16)       # 20 800 MAD : 20 fraudes manquees, 16 fausses alertes
... puis votre meilleur seuil, son cout, et ses deux compteurs
```

À rendre : le seuil trouvé, l'économie réalisée, et une phrase sur le prix de cette économie pour l'équipe qui traite les alertes.

Défi 3 — Le pipeline qui refuse les mauvaises données

Ajoutez une étape 0 qui vérifie les données avant toute préparation. Le squelette fournit un contrôle ; à vous d'écrire les autres, puis d'ajouter `valider` en tête de la liste des étapes de `pipeline.py`.

src/valider.py + fichier corrompu

```
cat > src/valider.py << 'EOF'
import sys, pandas as pd
fichier = sys.argv[1] if len(sys.argv) > 1 else 'data/transactions.csv'
df = pd.read_csv(fichier)
erreurs = []
if (df['montant'] <= 0).any():
    erreurs.append(f"montants <= 0 : {int((df['montant'] <= 0).sum())} lignes")
# A COMPLETER : colonnes attendues presentes, aucune valeur manquante,
# nuit dans {0, 1}, freq_24h >= 1, taux de fraude entre 1 % et 50 %
if erreurs:
    print('VALIDATION ECHOUEE'); [print(' -', e) for e in erreurs]; sys.exit(1)
print(f'validation OK : {len(df)} lignes, taux de fraude {df.fraude.mean():.1%}')
```

```
EOF
python - << 'EOF'
import pandas as pd, numpy as np                # fabrique un fichier volontairement abime
df = pd.read_csv('data/transactions.csv')
df.loc[df.sample(40, random_state=1).index, 'montant'] = -df['montant']
df.loc[df.sample(25, random_state=2).index, 'anciennete'] = np.nan
df.loc[df.sample(10, random_state=3).index, 'nuit'] = 3
df.to_csv('data/transactions_corrompu.csv', index=False)
EOF
python src/valider.py
python src/valider.py data/transactions_corrompu.csv
```

RÉSULTAT ATTENDU

```
validation OK : 5000 lignes, taux de fraude 11.4%

VALIDATION ECHOUEE
 - valeurs manquantes : 25
 - montants <= 0 : 40 lignes
 - nuit doit valoir 0 ou 1
(une fois les contrôles complétés ; le code de sortie vaut 1)
```

Défi 4 — La reproductibilité croisée

Le binôme A pousse son dépôt sur GitHub. Le binôme B le clone, installe les dépendances de `requirements.txt` et lance `mlflow run .` sur sa propre machine. Le défi est réussi si les métriques obtenues sont identiques au chiffre près. Consignez tout ce qui a empêché la reproduction du premier coup : c'est le livrable le plus intéressant.

Questions du compte rendu

1. Pourquoi l'étape d'entraînement ne doit-elle jamais refaire elle-même le découpage train / test ?
2. Quel intérêt y a-t-il à structurer les exécutions en parent et enfants plutôt qu'en une seule ?
3. Le seuil de validation est fixé à 0,82. Comment le choisiriez-vous dans un vrai projet ?
4. Que se passe-t-il si deux étudiants lancent le pipeline avec des params.yaml différents sans le versionner ?
5. Quelle étape du pipeline serait la plus coûteuse sur un vrai jeu de données, et que cela implique-t-il ?
6. Avec stratify=df['fraude'], le F1 passe de 0,8636 à 0,7849. Le modèle est-il devenu moins bon ? Que mesure-t-on réellement ?

Livrables

- Les fichiers src/preparer.py, src/entrainer.py, src/evaluer.py, pipeline.py, params.yaml et MLproject
- Capture de l'exécution parente MLflow avec ses trois exécutions enfants, et capture de l'exécution refusée
- Compte rendu : schéma du pipeline et réponses aux six questions
- Pour chaque défi tenté : le code, la sortie obtenue et trois lignes d'analyse

Barème

Critère	Points
Découpage en trois étapes indépendantes	5
Paramètres entièrement externalisés dans params.yaml	3
Exécutions parente et enfants, artefacts journalisés	4
Règle de validation démontrée dans les deux cas	4
Pipeline rejouable par mlflow run, compte rendu	4
Bonus défi 1 : contraintes respectées +1, podium sur le jeu caché +1	+2
Bonus défis 2, 3 et 4 : 0,5 point chacun, analyse comprise	+1,5
Total (bonus plafonné à 3 points, note plafonnée à 20)	20